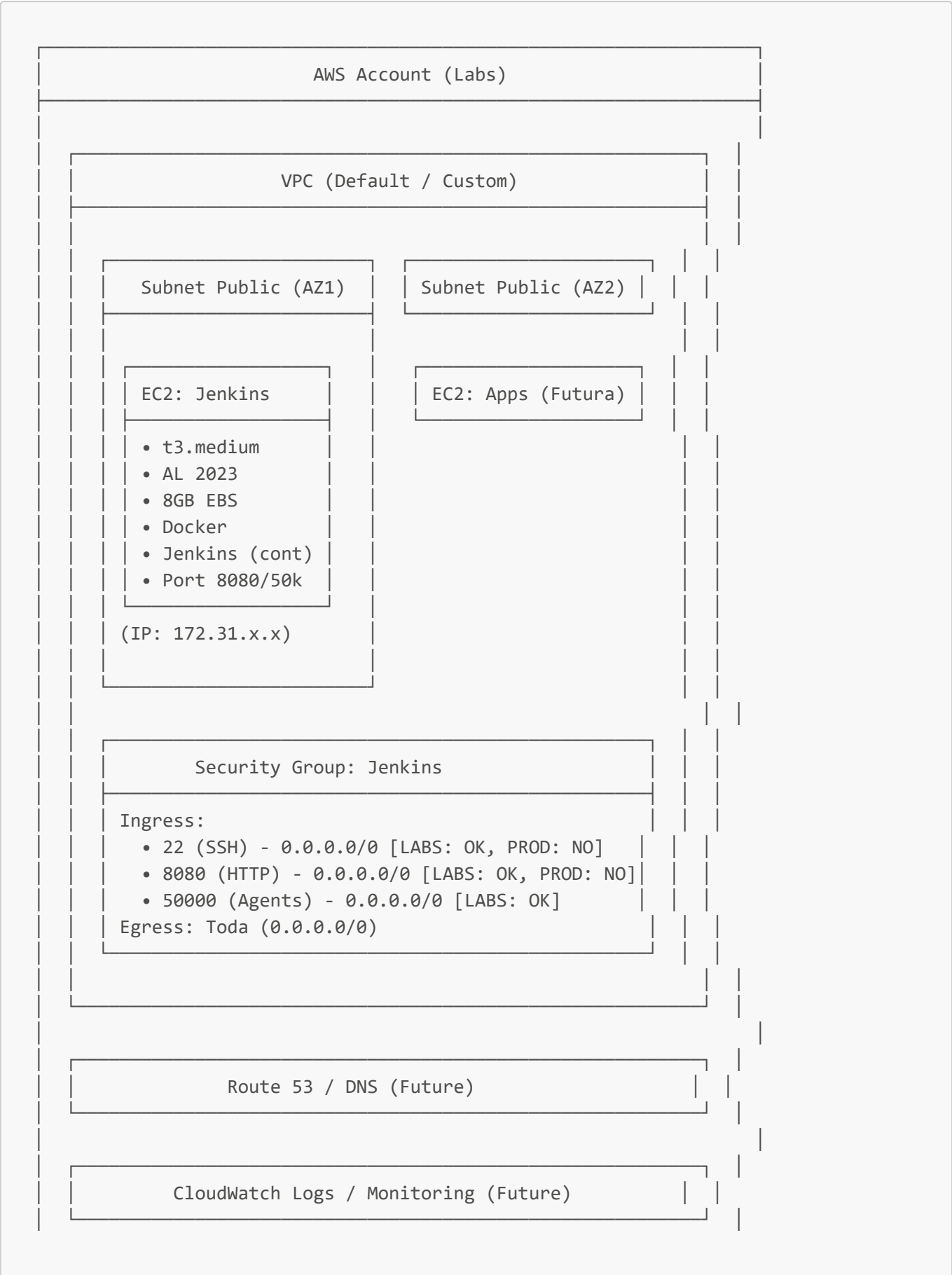


Arquitectura de SecureNet Cloud

Diagrama de Infraestructura



Capas de Aplicación

Layer 1: Infrastructure (IaC)

Terraform Files:

- Defines EC2, Security Groups, IAM roles, Networks
- Manages SSH keys, storage, security policies
- Enables reproducibility and versioning

Layer 2: Bootstrap (user_data)

Shell Scripts:

- `jenkins_app.sh`: Instala Docker, Docker Compose, levanta Jenkins
- `docker_app.sh`: (Futuro) Instala deps de aplicaciones
- Se ejecutan en primer arranque de EC2

Layer 3: Containerization (Docker)

Jenkins Container:

- Image: `jenkins/jenkins:lts`
- Ports: 8080 (web), 50000 (agents)
- Volumes: `jenkins_home` (persistent)
- Healthcheck: HTTP GET a `/login`

Apps Containers (Futuro):

- según microservicios del proyecto
- Orquestados con Docker Compose

Layer 4: Orchestration (Docker Compose)

Jenkins Compose:

```
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8080/login"]
      interval: 30s
      timeout: 10s
      retries: 10
```

```
ports:
  - "8080:8080"
  - "50000:50000"
volumes:
  - jenkins_home:/var/jenkins_home
```

Layer 5: Management (Power Apps)

Future tooling:

- CLI / Dashboard para gestión de contenedores
- Inicio/parada/reinicio de servicios
- Monitoring y alertas
- Backup y disaster recovery

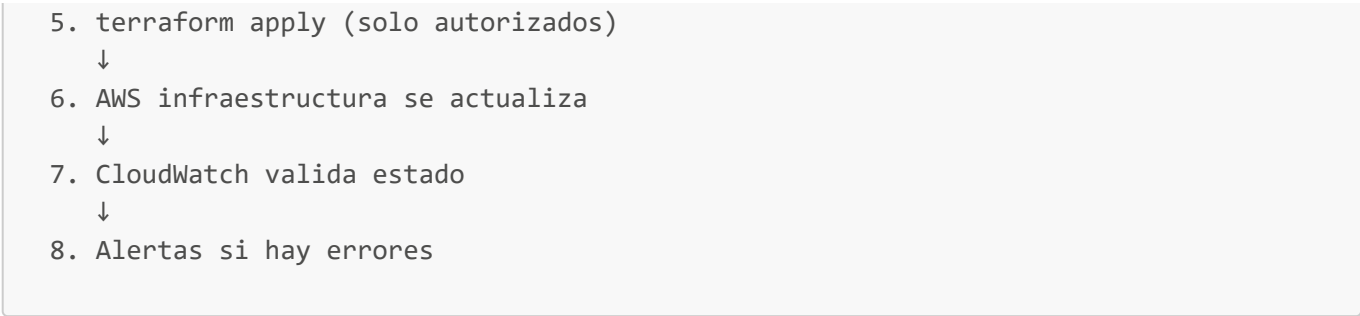
Flujo de Cambios

Ciclo Local → Cloud

1. Dev cambios en código
- ↓
2. Commit a Git (rama feature/terraform)
- ↓
3. Jenkins CI pipeline (optional)
- ↓
4. Merge a main
- ↓
5. Tag de release
- ↓
6. Jenkins CD pipeline
- ↓
7. Deploy a EC2 Apps
- ↓
8. Verificación de healthchecks
- ↓
9. Alertas si falla

Ciclo de Infraestructura

1. Cambios a Terraform (.tf files)
- ↓
2. Commit a Git (rama feature/terraform)
- ↓
3. terraform plan (verificar cambios)
- ↓
4. Code review (GitHub PR)
- ↓



Diferencias: Labs vs Producción

Aspecto	Labs	Producción
AWS Account	Cuenta laboratorios	Cuenta oficial
SSH Access	0.0.0.0/0	Bastion o VPN limitada
HTTP 8080	0.0.0.0/0 (OK para APIs)	Load Balancer + HTTPS
Backend Terraform	Local (<code>terraform.tfstate</code>)	S3 + DynamoDB (locked)
Instance Type	t3.medium	t3.medium o mayor según load
Auto-Scaling	No	Sí (ASG)
RDS	No	Sí (Multi-AZ)
Backups	Manual	Automated (Snapshots)
Monitoring	Básico (CloudWatch)	Completo (Prometheus+Grafana)
High Availability	No	Multi-AZ
Disaster Recovery	Manual	Automated

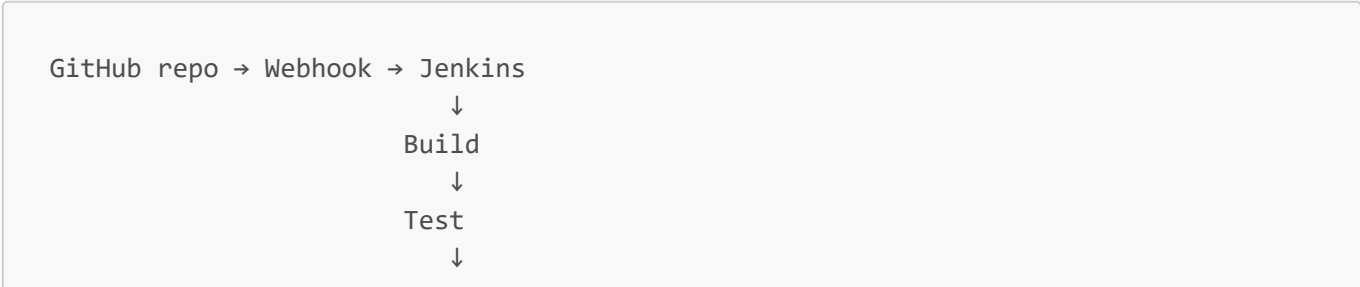
Integración con DevOps

Jenkins dentro de SecureNet

Purpose:

- Central CI/CD orchestration
- Deploy automation
- Pipeline management

Integración esperada:



```
Deploy to EC2:App
↓
Healthcheck
↓
Notify
```

Power Apps como extension

```
Power Apps → Power Automate → Gestion basica de contenedores
↓
Docker CE
↓
Containers (Apps, Monitoring, etc.)
```

Escalabilidad (Roadmap)

Fase 1: Single deployment (Current)

- 2 EC2 (Jenkins + Apps)
- Volumes locales

Fase 2: Multi-environment

- Labs (staging)
- Production
- Disaster Recovery

Fase 3: Auto-scaling

- Auto Scaling Groups (ASGs)
- Load Balancers (ALB/NLB)
- RDS Multi-AZ

Fase 4: Kubernetes (optional)

- EKS (Elastic Kubernetes Service)
- Helm charts
- Service mesh (Istio/Linkerd)

Security Layers

Layer 1: AWS Account Level

- Root account protection (MFA, no API keys)
- IAM roles with least privilege
- Organizations for multi-account setup

Layer 2: Network Level

- VPC security (private/public subnets)
- NACLs (stateless firewall)
- Security Groups (stateful firewall)

Layer 3: Instance Level

- OS hardening (Amazon Linux best practices)
- SSH key-pair (no password logins)
- Non-root user (ec2-user)

Layer 4: Container Level

- Image scanning (Amazon ECR scanning)
- Read-only filesystem
- Resource limits (CPU, memory)

Layer 5: Application Level

- Secrets management (AWS Secrets Manager)
- Encryption in transit (TLS)
- Encryption at rest (EBS encryption)

Troubleshooting

Jenkins no inicia

```
# SSH a la instancia
ssh -i <key> ec2-user@<ip>

# Ver logs de user-data
cat /var/log/user-data.log

# Ver estado de Docker
sudo docker ps
sudo docker logs jenkins

# Ver healthcheck
cat /opt/jenkins/.user_data_ran
```

Acceso SSH denegado

- Verificar security group permite puerto 22
- Verificar IP pública en AWS console
- Verificar permisos de clave privada (`chmod 600`)

Contenedor en estado unhealthy

```
sudo docker ps
sudo docker inspect <container>
sudo docker logs <container>
```

Políticas de Cambio

Cambios de Terraform

1. Nunca aplicar directo en prod (usar PR reviews)
2. Siempre hacer `terraform plan` primero
3. Documentar cambios en commit message
4. Usar `terraform import` para recursos externos

Cambios de Scripts

1. Testar en laboratorios primero
2. Usar `-replace` para aplicar en instancias vivas
3. Mantener backups de configuraciones anteriores
4. Comunicar cambios al equipo

Métricas de Éxito

- ☒ Infraestructura desplegada y validada
- ☒ Jenkins operativo con healthcheck passing
- ☒ SSH acceso funcional
- ☒ Logs centralizados y accesibles
- ☒ Roadmap de Power Apps documentado
- ☒ Runbook de disaster recovery operativo